Ассистент Порфирий

Исправление 2: один заход

Архив porfiry-fix-2.zip: новый контроллер чата и проверка, которая сама пишет вывод

Что в архиве

Файл	Зачем
`apply.sh`	Ставит исправление: копия старых файлов, новые файлы, права, кэши, перезапуск служб
`rollback.sh`	Откат. Точную команду печатает `apply.sh` в конце
`porfiry-selfcheck.php`	Проверка чата: модель напрямую, чат через Apache, журнал сервера. Печатает ИТОГ и ПРИЧИНА
`porfiry-check.sh`	Проверка ресурсов сервера — та же, что в диагностике 2
`site/…/ChatController.php`	Новый контроллер чата
`site/…/ChatTest.php`	Тесты под новый контроллер
`README.txt`	Три команды для запуска — их видно прямо на сервере: `cat README.txt`

Что меняется

- Контроллер больше не прерывает ответ, когда PHP решает, что браузер ушёл (`connection_aborted()`). Это то же, что делала ваша правка `sed` из диагностики 1, — новый файл её заменяет.
- На каждый ответ сервер пишет одну строку в `storage/logs/porfiry.log`: чем кончился ответ, сколько кусков ушло в браузер, видит ли PHP обрыв связи и скорость модели. Текст вопросов не пишется.
- Ответ заканчивается строкой `{"done":true}` — так проверка отличает «дошёл целиком» от «оборвался». Файл `chat.js` менять не нужно.

> Пока идёт проверка, кнопка «Стоп» и закрытая вкладка не останавливают ответ на сервере — он дописывается до `LLM_NUM_PREDICT` токенов. Остановку вернём, когда станет ясна причина обрыва.

Шаг 1. Архив на сервер

Перенесите `porfiry-fix-2.zip` на сервер в домашнюю папку (по FTP или с флешки) и распакуйте:

```
cd ~
python3 -m zipfile -e porfiry-fix-2.zip .
cd ~/porfiry-fix-2
SITE=/путь/к/сайту
```

Вместо `/путь/к/сайту` — папка, где лежит `artisan`. Если распаковали на Windows и залили папку `porfiry-fix-2` целиком — просто перейдите в неё командой `cd`.

Шаг 2. Поставить

```
sudo bash apply.sh "$SITE"
```

Ожидается четыре строки `OK` и в конце две готовые команды: проверка и откат. Строка `ОШИБКА` — перепишите её.

Шаг 3. Проверить

```
sudo php porfiry-selfcheck.php "$SITE" http://127.0.0.1
```

Второе — адрес сайта с самого сервера, как в `check-site.py --base`. Проверка спрашивает модель дважды: напрямую и через сайт. На медленной машине это 3–10 минут — дождитесь строки `ИТОГ:`.

Сразу после этого задайте в чате в браузере тот же вопрос: «Ответь одним коротким предложением: что такое атом?». Когда ответ закончится, посмотрите последнюю строку журнала:

```
tail -1 "$SITE"/storage/logs/porfiry.log
```

Что значит результат

Что видно	Что это значит и что делать
`ИТОГ: чат работает`, в браузере ответ целиком	Готово. Если медленно — раздел «Скорость».
`ИТОГ: чат работает`, а в браузере одна буква	Сервер исправен — ответ режет что-то между сервером и вашим ПК: прокси или антивирус с проверкой веб-трафика. Добавьте адрес сервера в их исключения. В строке `tail -1` после вопроса из браузера будет «PHP видит обрыв связи после куска 1».
`ПРИЧИНА: … рвёт Apache`	PHP отправляет ответ, но Apache его не доносит. Нужен следующий шаг — см. последний раздел.
`ПРИЧИНА: PHP перестал получать поток от Ollama`	Ollama оборвала ответ. Сфотографируйте вывод `sudo journalctl -u ollama -n 30 --no-pager`.
`ПРИЧИНА: модель упёрлась в лимит длины`	В `.env` проверьте `LLM_NUM_PREDICT=1000` и `LLM_NUM_CTX=4096`, затем команды из раздела «Скорость» без строки `sed`.
`ПРИЧИНА: PHP не дописал журнал`	PHP прервали посреди ответа. Сфотографируйте `sudo tail -20 /var/log/apache2/error.log` и `sudo tail -20 /var/log/php8.1-fpm.log`.
`OLLAMA: не отвечает`	`sudo systemctl status ollama` — должно быть `active`. Затем шаг 2 ещё раз.
`КОНТРОЛЛЕР: старая версия`	Шаг 2 не сработал или в `SITE` другой путь.
`САЙТ: … не открылась`	Неверный адрес во втором аргументе — укажите тот, что в `check-site.py --base`.
`ЖУРНАЛ: …` с «исключение» или «ошибка Ollama»	Перепишите текст после этих слов.

Скорость

Строка `СКОРОСТЬ` — сколько токенов в секунду модель пишет на этой машине. Проверочный файл из диагностики 2 получил 200 строк за 240 секунд — около 1 токена в секунду. Для этой модели нормально 5–10 ток/с на 4 ядрах с AVX2.

При 1 ток/с ответ в 1000 токенов идёт около 17 минут, а два параллельных ответа делят процессор. 30 человек при такой скорости ждут десятки минут — это предел машины, а не ошибка сайта.

- Короче ответы — быстрее очередь. Поставить 400 токенов:

```
cd "$SITE"
sudo sed -i 's/^LLM_NUM_PREDICT=.*/LLM_NUM_PREDICT=400/' .env
grep NUM_PREDICT .env
sudo -u www-data php artisan config:cache
sudo systemctl restart apache2
sudo systemctl try-restart php8.1-fpm
```

- Главное средство — больше ядер виртуалке и тип процессора host-passthrough в настройках гипервизора: виртуалка получает все инструкции процессора хоста.
- Если ядер не добавить — модель поменьше (Qwen3-1.7B): примерно вдвое быстрее, но слабее. Её нужно принести файлом.

Ресурсы сервера

Если всё тормозит или ведёт себя странно — та же проверка ресурсов, что в диагностике 2:

```
sudo bash ~/porfiry-fix-2/porfiry-check.sh
```

Тесты (по желанию)

`apply.sh` уже положил тесты под новый контроллер. Прогнать:

```
cd "$SITE"
sudo -u www-data php artisan config:clear
sudo -u www-data php artisan test
sudo -u www-data php artisan config:cache
```

Откат

Команду с нужными путями печатает `apply.sh` в конце. Её вид:

```
sudo bash ~/porfiry-fix-2/rollback.sh "$SITE" ~/porfiry-fix-2/backup-ДАТА
```

Что сообщить разработчику

- Вывод проверки: строку `ИТОГ`, строки `ПРОБЛЕМА` и `ВНИМАНИЕ`, строки `СКОРОСТЬ` и `ЖУРНАЛ`.
- Что показал браузер и строку `tail -1` после вопроса из браузера.
- Только если `ПРИЧИНА: … рвёт Apache` — вывод команд:

```
cd /etc/apache2
sudo grep -rn "fcgi\|flush\|Timeout\|deflate" sites-enabled conf-enabled
sudo tail -20 /var/log/apache2/error.log
```